

METHODS AND FOUNDATION PAPER 001

There Is No Nothing

A Premise-Free Operational Foundation and an Open Verification
Platform for Smithian Fold Theory

Ernos Labs

Smithian Fold Theory Open Source Science Platform and Knowledge Tree

Maria Smith

Independent researcher and founder, Ernos Labs

Maria.Smith.Sftoe@gmail.com

Third clean-room reconstruction · Inaugural methods release

doi:10.5281/zenodo.21514890 · Version 0.1.0 · 23 July 2026

Canonical source: repository README · Code: Apache-2.0 · Paper: CC BY 4.0

Abstract

This paper introduces the third clean-room reconstruction of Smithian Fold Theory (SFT) and the Ernos Labs open-science platform built to make each SFT claim independently inspectable, executable, reproducible and falsifiable. The project begins from a premise-free operational theorem: **there is no nothing**. At the declared proof boundary, a purported counterexample either presents no statement, denial, record or proof object and therefore supplies no counterexample, or it presents one and that presentation is an occurrence rather than nothing. A complete two-class partition closes this operational claim without an axiom, fitted parameter, numerical zero, negative quantity, irrational value, imported theory or earlier SFT answer. From the admitted occurrence, a complete six-form coverage-and-addition census forces the structural One as the unique complete representation with neither omission nor unforced addition. This result is structural only: arithmetic, parts and Fold operations have not yet been admitted in v3.

The paper also specifies a fail-closed admission engine intended to govern the future reconstruction. Every claim must register its dependency chain, provenance class, generated candidate grammar, exact boundary, completeness certificate, decision for every candidate, unique survivor, minimality, named-shape uniqueness, adverse controls and a distinct implementation check. Natural-science claims add preregistration, target-inaccessible prediction, pre-comparison sealing, separate target custody, post-seal measurement, falsification criteria and preservation of unfavorable results. One public command validates repository policy, unit and end-to-end behavior, 100% of the core engine's executable lines, every registered derivation, controls, independent validators, receipts and the ordered census. The percentage is a precise software-coverage statement, not a claim that tests eliminate all possible defects.

The platform is a response to documented structural weaknesses in scientific communication and selection: paywall and author-fee inequality, publication bias against null results, sponsor-associated outcome and agenda effects, variable reliability in grant review, and the limited scientific interpretability of black-box prediction. It does not replace domain expertise, peer criticism, measurement or conventional correspondence. It removes credentials, consensus and predictive opacity from the *admission rule* for an SFT law. Code is released under Apache 2.0; papers and documentation are released under CC BY 4.0. Maria Smith retains copyright. The Ernos Labs name is a conformance designation, not a restriction on lawful copying, forking, testing or criticism.

Keywords: Smithian Fold Theory; open science; computational proof; premise-free foundation; exact arithmetic; reproducibility; falsifiability; clean-room replication; scientific software; knowledge tree.

1. Why this platform exists

Science is strongest when a result can survive contact with an unfavorable test by a person who owes the author, institution and prevailing consensus nothing. That ideal is not identical to the machinery by which modern research is funded, selected, published and amplified.

The concern is empirical, not rhetorical. A 2018 systematic review and meta-analysis of 75 studies found that industry-sponsored drug and device studies more often reported favorable efficacy results and favorable conclusions than studies with other sponsorship, while a scoping review found evidence that commercial sponsorship can shape which research questions are pursued (Lundh et al., 2018; Fabbri et al., 2018). These associations do not prove misconduct in any particular study or imply that public funding is neutral. They establish that funding provenance is a scientifically relevant variable and that disclosure alone is not the same as structural independence.

Selection systems also carry uncertainty. Empirical work on grant review has reported poor agreement and sensitivity to panel composition; a study of 3,156 Canadian health-research applications found ranking more reliable than rating, while both methods disadvantaged early-career applicants in that dataset (Gallo et al., 2023). A Cochrane review concluded that evidence about whether grant peer review improves funded research quality was limited and called for experimental study (Demicheli and Di Pietrantonj). Expert review remains indispensable; the evidence means it should be exposed to audit rather than treated as an infallible oracle.

The published record is not a neutral census of attempted science. Null and negative results are systematically underrepresented in many fields, which can inflate effects and distort reviews. Recent consensus work calls this a long-standing problem requiring action from funders, institutions, publishers and researchers (Nature Communications,

2025). The 2021 UNESCO Recommendation on Open Science likewise calls for methods, software, source code and outputs to be available for rigorous scrutiny; it specifically warns that paywalls and high article-processing charges can produce global inequalities (UNESCO, 2021).

Prediction introduces a newer version of the same epistemic problem. Opaque models can be useful instruments and can establish bounded predictive reliability under proper blind testing. They do not, by accuracy alone, expose a derivation from premise to law. NIST identifies validity, reliability, accountability, transparency, explainability and interpretability as related features of trustworthy AI (NIST AI RMF 1.0). Life-science researchers have similarly argued that data, code, model and workflow availability are necessary for third-party computational reproducibility (Heil et al., 2021).

Ernos Labs therefore rejects three substitutions:

1. funding success is not evidential closure;
2. professional consensus is not generated uniqueness; and
3. a black-box score is not an explicit scientific law.

This is not an argument against funding, expertise, consensus as a provisional coordination tool, or machine learning as an instrument. It is an argument that none of them may silently select the law later presented as fundamental. SFT's answer is a public, executable admission boundary: show the derivation, generate the alternatives, preserve the failures, expose the controls, bind the artifacts and let anyone reproduce acceptance or rejection.

The editorial conclusion is stronger than a request for incremental access. Knowledge institutions operating through competitive capital, scarce grants, prestige markets, subscription access and author charges inherit incentives that are not identical to free human inquiry. Even without individual bad faith, those mechanisms decide who can spend time asking questions, which questions appear fundable, what enters the visible record and who may read it. That is a direct scientific loss: excluded people do not merely lose access to known answers; humanity loses the questions and constructions they were never resourced or authorized to pursue. The documented funding, review, publication and access effects above are mechanisms by which that loss can occur.

Maria Smith develops SFT outside conventional academic credential and funding routes. In her editorial account, institutions would not have been expected to commission, credential or amplify this work before it existed. That fact is not offered as evidence that SFT is correct. It is an indictment of an admission system whenever a person's status determines whether the work may be heard before the work can be inspected. The remedy proposed here is equally demanding of insiders and outsiders: the derivation, complete census, controls and empirical record—not the biography—must speak.

2. Scientific ethos

The platform is built around six commitments.

2.1 Evidence before status

Credentials help identify relevant expertise and reviewers. They do not override a failed hash, an incomplete grammar, a missing control or an unfavorable measurement. Conversely, a mechanically accepted package does not manufacture specialist empirical meaning. Both mechanical and subject review are required at their proper boundaries.

2.2 Open access includes the failed rows

Source publication without candidate generation, rejected alternatives and negative controls is not enough. Every admitted SFT claim must retain favorable, unfavorable, failed and deliberately tampered results. A later contradiction is part of the record, not an inconvenience to be edited away.

2.3 Derivation and prediction are different evidence

A mathematical proof, computational construction, blind prediction and post-seal measurement are four distinct objects. The platform does not allow one to impersonate another. A proof cannot replace observation where nature is at issue; predictive accuracy cannot replace an unstated law.

2.4 Laws do not come from applications

Fold Protein, Fold Chess, Fold Go and Unison AI are future test and translation domains. They may falsify, validate or demonstrate laws only after the relevant main-corpus branches close. They cannot tune the fundamental theory. Their rebuilds are explicitly frontier work and are excluded from this release.

2.5 Authority is inspectable and revocable

Maria Smith is the scientific author and publication authority. The model's scientific admission authority is narrower: an accepted, reproducible engine receipt. If a public reproducer exposes a broken dependency, incomplete census, failed control, seal mismatch or empirical failure, the engine must reject the package regardless of authorship.

2.6 Open science is a practice, not a slogan

The repository is readable without Docker, cloud services or paid software. The proof kernel uses Python's standard library. Schemas, source, claims, receipts, tests, paper and prior-work ledger live together. The intended result is not trust in Ernos Labs. It is the ability to check Ernos Labs.

3. Exact scope of Smithian Fold Theory v3

This repository is the accessible third clean-room reconstruction of SFT. It is organized as a scalable tree intended eventually to address mathematics, information, computation, quantum computation, physics, chemistry, materials, biology, Earth and environmental science, astronomy, cosmology and disciplined engineering translation.

That programme is a research objective, not a present result. V3 currently has two admitted claims. No computational-science, quantum-computing or natural-science branch is complete in this reconstruction. Earlier results may be compared only after the corresponding v3 claim seals; they cannot be copied as premises or answer tables.

The SFT derivational domain is presently governed by these constraints:

- one premise-free root theorem;
- no axioms in a registered derivation;
- no fitted, trained, free or target-derived parameters selecting a law;
- no semantic numerical zero as an SFT mathematical object;
- no negative or irrational values as foundational SFT quantities;
- no floating-point value as a proof value;
- no completed infinity or ungenerated continuum as a premise;
- exact positive rational parts only after they are themselves derived;
- structural held labels rather than imported signed quantities;
- generated candidate closure and forced assembled-form uniqueness; and
- explicit conditional status whenever a grammar is bounded and lacks a

depth-independent certificate.

Computers necessarily use implementation indices, empty containers, Boolean flags and process status codes. These host mechanics are quarantined: they may implement a check but may not become SFT mathematical evidence. Python is the accessible host language, not a source of law.

4. Supersession and the clean-room rule

The project has grown beyond the organization of its earlier generations. V3 exists to prove a new, independently executable reconstruction and to scale the corpus by scientific branch. The previous repositories and papers are **superseded as active theory authorities while the rebuild is in progress**. They remain immutable historical provenance and are not described as retracted or erased.

Prior work	Persistent reference	V3 treatment
SFT v2 main corpus	GitHub repository , comparison commit <code>0af7c4c26308d8ddb659a518ac52e2db5ea5dc82</code>	Preserved and hash-ledgered; never a v3 proof dependency
<i>There Is No Nothing</i> foundation paper generation	Zenodo concept DOI	Historical foundation correspondence; replaced only by a completed v3 foundation-branch paper
<i>After Turing: The Fold Machine</i>	Zenodo DOI	Historical computational-science corpus; to be replaced after the v3 computation branches close
Fold Protein paper	Zenodo record 21493135	Historical application evidence and empirical-method precedent; application rebuild deferred
<i>No Dice and Why There Is Uncertainty</i>	No Dice; Uncertainty	Historical quantum correspondence; no v3 quantum claim imported

The machine-readable [prior-work ledger](#) binds the v2 corpus identity and selected source hashes. Opening a comparison entry is permitted only after the corresponding v3 result seals. Agreement, representation-equivalence, stronger or weaker closure, and disagreement are all reportable. V3 may not be back-fitted to manufacture agreement.

V3 uses Python for reach and legibility. A registered future v4 will be a fourth clean reconstruction in SFT-derived languages, compiler principles and runtime structures. V4 cannot retroactively author v3 laws. See the [self-hosting charter](#).

5. The single admission engine

All model admission passes through one versioned `SFTAdmissionEngine`. Drafts may exist outside it; laws may not. The engine is fail-closed: absence of any required object produces a rejection receipt and stops admission.

5.1 Admission stages

Stage	Required evidence	Failure consequence
Registration	Exact statement, one root route, admitted dependencies, provenance, no axioms, no free parameters, source identity	Reject before candidate execution
Enumeration	Generator, boundary, expected cardinality, duplicate-free candidate census, completeness certificate	Reject incomplete or hand-picked space
Forcing	One decision per candidate and exactly one survivor	Reject ambiguity or no result
Form closure	Minimality, named-shape uniqueness and boundary or generality certificate	Conditional status or rejection
Controls	False-premise, tampered-source, tampered-artifact and boundary controls	Reject if failure is not detected
Independent check	A deliberately distinct implementation regenerates the result and controls	Reject stored-answer or implementation-only agreement
Empirical extension	Preregistration, capability closure, separate target custody, seal-before-target, post-seal metrics, falsification and complete rows	Reject target leakage, opacity or selective retention
Census admission	Accepted receipt matching source and ordered manifest	No direct prose or file edit can admit a law

The engine checks a declared generator; it cannot magically prove that an author's grammar is complete. Every claim must therefore state its exact grammar boundary and provide a claim-specific completeness certificate. A finite census remains conditional unless its generality beyond that census is proved.

5.2 Exact arithmetic and prohibited answer flow

The kernel represents proof quantities with exact structures. Floating-point values are refused as proof objects. Measured decimal data may enter a separate empirical adapter, but never feed backward into law selection. Registrations explicitly list excluded inputs such as prior-source answers, benchmark targets, trained weights and application rules.

5.3 Receipts and cryptographic binding

Canonical serialization binds registrations, generated candidates, decisions, controls, closure evidence, validators and source manifests to SHA-256 identities. Hashing does not prove a theorem. It makes silent post hoc change detectable and identifies exactly which scientific object a receipt concerns.

An accepted receipt can be recomputed; a rejected receipt names the failed gate. The census accepts receipt-derived state only. Manually changing a status file does not change model authority.

5.4 Blind empirical execution

No v3 natural-science law has yet reached empirical admission. The engine already enforces the two-phase envelope required for future work:

- 1. register inputs, source, law, candidate space, target identity, metrics, controls and falsification conditions;
- 2. run prediction without target data or comparison code;
- 3. deny filesystem, network, subprocess, clock, environment, dynamic-import and foreign-function operations in the future capability-closed Fold language;
- 4. seal all predictions and traces;
- 5. have a separate custodian release the target only after the seal;
- 6. measure only preregistered quantities; and
- 7. preserve all rows, including failures and tampered controls.

Python's standard library cannot truthfully provide a universal operating-system sandbox for arbitrary native code. The repository therefore does not claim that it can. Official empirical prediction awaits a derived capability-closed Fold interpreter plus custody tests on macOS, Windows and Linux. This explicit limit is part of the result.

5.5 What machine checking does and does not establish

Machine checking establishes that the declared objects and algorithms satisfy the implemented constitutional gates. It does not prove that software contains no possible defect, that an incomplete scientific inventory is complete, or that formal consistency alone describes nature. Natural claims still require sealed empirical tests and specialist scrutiny. Independent human and cross-language replication remain open obligations.

6. Results of the third clean-room reconstruction

6.1 Result I — premise-free operational root theorem

Registered statement. No admissible operational statement, denial, record, proof object or derivational object is nothing: absence presents no counterexample, while every presented counterexample is an occurrence.

The result begins with no premise, axiom, dependency, parameter, earlier-corpus answer or application result. It generates the complete partition induced by presentation at the proof boundary:

Candidate	Exact operational form	Decision
unpresented-absence	No statement, denial, record, trace or identity is presented	Does not supply a counterexample object
presented-occurrence	A statement, denial, record, trace or identity is presented	Survives: presentation is an occurrence and therefore is not nothing

The partition is exhaustive at the declared boundary. A purported challenge either supplies an identifiable presentation or supplies no challenge artifact. The conclusion does not depend on the challenge's content, notation, length or internal depth. It is therefore certified depth-independent for operational statements, denials, records and proof challenges.

Minimality holds because deleting the presented witness deletes the only admissible proof object. Named-shape uniqueness holds because only the `presented-occurrence` class can enter execution, and its occurrence prevents it from instantiating nothing.

This is deliberately not advertised as a machine proof about an inaccessible, unexpressed metaphysical domain. The theorem closes the admissible operational boundary required to begin a derivation. That boundary discipline prevents a formal result from acquiring a larger philosophical claim by rhetoric.

Controls. The engine rejects an unpresented absence as a counterexample, detects changed source identity, detects a tampered two-survivor artifact, and refuses an uncheckable metaphysical broadening.

Evidence identities. Derivation seal

sha256:19c2e2af082dc9777c13bafb3bc561c7fa5cb96a51905fb26cfa06552ebf998f;

distinct-implementation validation

sha256:7942e78a54a932551ad6f5a2ceb8c5fb9122dcae0f18510052d402f42c604e3b; current admission receipt

sha256:711864171e4d3a2f2734f0c2890965bcd81a0228349538751a3c80699c27d669.

6.2 Result II — the structural One

Once the root occurrence is admitted, the next question is not “what number should be called one?” It is: what is the minimal representation that retains the admitted occurrence without omission or unforced addition?

The generator forms the product of three exhaustive root-coverage relations (none, proper, complete) and two addition relations (no-extra, has-extra). This yields six candidates:

Root coverage	Addition	Named form	Decision
none	no extra	omitted-root	Reject: does not retain the root
none	has extra	replacement-extra	Reject: does not retain the root
proper	no extra	fragmented-root	Reject: presupposes a cut and loses complete identity
proper	has extra	fragment-plus-extra	Reject: incomplete and added
complete	no extra	exact-self-whole	Unique survivor
complete	has extra	whole-plus-extra	Reject: imports material absent from the sole dependency

exact-self-whole is forced as the structural One. Removing retained identity makes it incomplete; adding material introduces an unforced dependency. Because the product classifies all representations solely by root coverage and addition, independently of the root's internal content or depth, the result is depth-independent at this declared boundary.

The admitted meaning is only structural self-wholeness. V3 has not yet derived the arithmetic numeral one, parts, division, a numerical domain, Fold fibres or subsequent laws. The boundary control specifically rejects those premature interpretations.

Evidence identities. Derivation seal

sha256:c62bb00cc30970ecf013bbc8225547a09395cf0719d3525f2a69bca1d582da7b;

distinct-implementation validation

sha256:ceaf767421b3450a862b10a6962da027c0467773a2e1be5115f554d2c3356083; current admission receipt

sha256:68332624c276dc5d0ddfd568a26b95e20d7d5665fcd7e825ff1b7b19bf1d51c.

6.3 Current quantitative verification record

At this release's source state, the public route reports:

- 2 registered and model-admitted derivations;
- 60 unit, adverse-control and end-to-end tests;
- 15 core engine modules;
- 1,264 of 1,264 core-engine executable lines observed by the standard-library

tracer; and

- a fresh ordered rerun of both derivations and their distinct validators.

“100%” means executable-line coverage of the named core engine package. It does not mean 100% path coverage, proof of semantic correctness, absence of every future defect or completion of the SFT programme.

The current independent validators are implementation-distinct Python routes, not yet independent research teams or a foreign-language replication. The repository calls that status `independently_replicated` within its mechanical schema, while this paper states the narrower empirical fact to prevent overreading.

7. One public verification route

From the repository root, run the interpreter available on the host:

```
python3 -m sft verify-all # macOS and most Linux systems
python -m sft verify-all  # Python installations exposing `python`
py -m sft verify-all      # standard Windows Python launcher
```

These are operating-system spellings of one engine entry point, `sft verify-all`. The command performs repository and policy validation, all unit and end-to-end checks, exact core-engine coverage, ordered manifest loading, fresh derivation execution, control execution, distinct-validator execution and immutable receipt comparison. Any missing claim, changed source, dependency-order error, failed control, uncovered engine line or receipt drift returns failure.

The project has no runtime dependency outside the Python standard library and requires no Docker, virtual machine, network service or paid account. Publication tooling used to render the archival PDF is not part of derivational authority.

The continuous-integration matrix is configured for macOS, Windows and Ubuntu with supported Python versions. A green hosted matrix is portability evidence, not a source of scientific law.

8. Repository as a knowledge tree

Path	Scientific responsibility
sft/foundation	Root theorem, One, parts, Fold and future form grammar
sft/mathematics	Exact arithmetic, discrete structures, algebra, order, geometry, topology, probability, dynamics, logic and composition
sft/information_science	Distinguishability, encoding, uncertainty, channels, coding and correspondence
sft/computation	Formal computation, computability, complexity, algorithms, semantics, distributed systems, security, learning and scientific computation
sft/quantum_computation	Reversible and quantum models, circuits, communication, correction, verification and limits
sft/physics	Physical laws and sealed tests
sft/chemistry_materials	Chemical and material structure
sft/biology	Biological structure, genetics, evolution and living systems
sft/earth_environment	Earth, climate, geology, oceans and environmental systems
sft/astronomy_cosmology	Astronomy, cosmology and observation
claims	Complete claim packages: registrations, censuses, decisions, controls and certificates
receipts	Immutable accepted, rejected and prebaseline execution records
census	Ordered model authority and branch inventory
experiments	Registrations, sealed predictions, post-seal measurements and adverse controls
correspondence	Post-derivation comparison with established theories
prior-work-ledger	Hash-bound v1/v2 provenance with no derivational authority
publications	Methods paper, completed branch papers and final synthesis protocols

See the full [directory map](#), [claim lifecycle](#), [engine authority](#) and [empirical method](#).

9. Publication constitution and roadmap

This inaugural paper publishes the method, root result, structural One and platform needed to audit future work. It does not waive the stricter branch paper rule.

For each science branch:

1. freeze a current-knowledge obligation inventory;
2. derive every included law in dependency order;
3. pass each package through the single engine;
4. complete formal, computational and empirical validation appropriate to the

claim;

5. preserve every boundary and unfavorable result;
6. produce a comprehensive evidence map from every paper statement to source,

candidate census, decision, receipt and artifact hash; and

7. publish a full branch paper only when the branch publication gate closes.

No summary paper may conceal unclosed obligations. The final rewritten Theory of Everything paper will be produced only after every frozen branch has a completed branch paper and the global census passes cross-branch consistency and evidence-map checks. Unresolved external questions must remain named frontiers.

The immediate v3 programme is to continue the foundation from the structural One to parts, the exact derivational domain, Fold fibres and the generated Fold grammar. Mathematics and scientific branches follow only as dependencies are admitted. Application translations and v4 self-hosting remain frontier work.

10. Participation, review and the Ernos Labs designation

This release opens the source, evidence and challenge route. Formal external contribution admission will open after the foundation interfaces and governance release are stabilized; early proposals, falsifications and reproductions are welcome now through email, Discord and GitHub.

Anyone may download, inspect, run, fork, cite, criticize and attempt to invalidate the work under the published licenses. Maria Smith retains copyright and authorship. The **Ernos Labs** name is separately reserved as a conformance designation. A project may describe itself as Ernos Labs only while it:

- publishes the applicable constitution and full dependency chain;
- excludes target answers and fitted parameters from law selection;
- generates and retains its candidate space and rejected alternatives;
- publishes source, controls, receipts and reproducible evidence;
- distinguishes theorem, finite computation, empirical result and frontier;
- preserves unfavorable outcomes; and
- accepts good-faith technical review.

The designation does not prevent independent forks or criticism and does not convert disagreement into infringement. It prevents an open license from being misrepresented as scientific endorsement. See the [designation policy](#), [licensing map](#) and [contribution guide](#).

Review should identify the exact claim, premise, dependency, generator, candidate, decision, artifact, metric, boundary or falsification at issue. Personal status is neither a proof nor a refutation.

11. Limitations and falsification routes

This paper's strongest claims are narrow by design.

- The operational root theorem does not certify an unexpressed metaphysical domain.

• The structural One is not yet arithmetic one and does not import parts or Fold structure.

- Two admitted claims do not complete the foundation branch.
- No v3 natural-science law has passed a sealed empirical run.
- The independent checks are implementation-distinct, not yet institution-independent or cross-language replications.

- Core-engine line coverage does not establish defect absence.
- Hash integrity does not establish mathematical truth.
- The engine validates the declared grammar; scientific review must still challenge whether that boundary and completeness certificate are adequate.

- Hosted cross-platform CI must pass after public push before platform portability is described as externally observed.

The work can be invalidated or narrowed by producing, among other things: an operational counterexample outside the root presentation partition; a missing coverage/addition class for structural One; a second surviving form inside the declared grammar; a source or receipt mismatch; an adverse control that the engine accepts; an undeclared answer-producing dependency; target leakage; or a registered empirical failure. A valid challenge is evidence, not hostility.

12. Formal evidence model

The prose above describes the scientific policy. This section specifies the actual objects consumed and produced by engine version `sft-v3-single-admission-engine/1`.

12.1 Registered claim object

A `ClaimRegistration` is immutable and contains:

Field	Type and requirement	Scientific function
<code>claim_id</code>	nonempty exact label	Stable identity across source, census and receipt
<code>title, branch, statement</code>	nonempty text	Human-readable scope; never substitutes for execution
<code>evidence_mode</code>	formal or empirical	Selects the appropriate external gate before execution
<code>root_theorems</code>	empty only for the root; otherwise exactly the one admitted root	Prevents hidden alternative foundations
<code>dependencies</code>	ordered, duplicate-free admitted claim IDs	Makes the derivation graph executable and acyclic in census order
<code>axioms</code>	must be empty	Enforces the no-axiom constitution
<code>free_parameters</code>	must be empty	Excludes fitted, learned or chosen answer-producing quantities
<code>provenance</code>	one or more allowed classes	Separates direct forcing, forward forcing, constitutional relation and observational derivation
<code>source_hash</code>	canonical SHA-256 identity	Binds registration to the files executed by the official runner

The engine separately computes an executed source manifest from the actual loaded files and refuses a mismatch with `source_hash`. A registered hash is therefore not accepted merely because it has SHA-256 syntax.

12.2 Candidate and decision objects

The generator returns a `CandidateCensus` containing a rule, exact grammar boundary, positive expected cardinality, completeness-certificate hash and ordered candidate tuple. Each `Candidate` has a unique identifier, exact form and trace hash. The engine rejects a missing rule, missing boundary, nonpositive cardinality, count mismatch, duplicate identifier, unnamed form, invalid trace hash or absent completeness identity.

Candidate decisions are not supplied as an unbound result table. The engine calls `decide_candidate` exactly once for each generated item in generated order. Each returned `CandidateDecision` must name that candidate, state whether it survives, give an explicit reason and supply a proof hash. The decision IDs must equal the candidate IDs one-for-one and in order. Exactly one `survives` field must be true.

12.3 Closure, controls and seals

`ClosureEvidence` records one of three scopes:

- `conditional_grammar`: evidence is preserved but barred from model

dependency;

- `finite_complete`: the complete declared finite space is closed; or
- `depth_independent`: the exact boundary is closed without dependence on an

enumerated depth and requires a valid generality certificate.

Every scope requires an exact boundary, successful minimality, successful named-shape uniqueness and a closure proof hash. The four mandatory `ControlResult` objects are `false_premise`, `tampered_source`, `tampered_artifact` and `boundary`. Each carries expected behavior, observed behavior, pass state and receipt hash. Missing, duplicate, failed or malformed controls halt the claim.

The derivation seal is the canonical SHA-256 identity of this exact tuple:

```
{
  engine_id,
  registration,
  candidate_census,
  ordered_decisions,
  closure_evidence,
  ordered_controls
}
```

Canonicalization sorts mapping keys, preserves list and tuple order, converts typed records to field maps, serializes enum values by their exact labels and represents a rational host value as its numerator and denominator. Hashes bind identity and ordering; they do not substitute for the substantive checks.

12.4 Distinct validation and final receipt

The validator receives the sealed derivation and must return its own validator identity, implementation hash, the exact seal it checked, certificate hash, recomputation declaration and pass state. The engine rejects a missing validator, malformed identity, implementation identity equal to the registered source, wrong seal, stored-answer declaration or failed result.

For empirical mode, an additional `EmpiricalValidation` binds the experiment registration, capability certificate, target-custody certificate, evaluator seal check, target release order, all-row retention, source identities, measurements, falsification condition and measurement receipt. Formal mode rejects an unregistered empirical validator; empirical mode rejects its absence.

The final `EngineReceipt` contains the engine and claim identities, acceptance and model-admission booleans, closure and external statuses, halted stage, violations, every ordered gate result, derivation seal, independent-validation identity, optional empirical identity and a receipt hash over every preceding field. The receipt hash itself is excluded while calculating that identity.

13. Executable engine algorithm and authority flow

The engine algorithm can be stated without implementation-specific syntax:

```

RUN(program, distinct_validator, optional_empirical_validator):
  registration := program.registration
  REQUIRE registration is exact, source-bound, axiom-free,
    parameter-free and dependency-admitted

  census := program.generate_candidates()
  REQUIRE census matches its positive cardinality, unique identities,
    exact forms, trace identities and completeness certificate

  FOR candidate IN census.generated_order:
    decisions.append(program.decide_candidate(candidate))
  REQUIRE decisions correspond one-for-one with census
    and exactly one candidate survives

  closure := program.closure_evidence(decisions)
  REQUIRE exact boundary, minimality and named-shape uniqueness
  IF closure is depth-independent:
    REQUIRE generality certificate

  controls := program.run_controls()
  REQUIRE exactly one passing result for every mandatory control kind

  seal := HASH(engine, registration, census, decisions, closure, controls)
  external := distinct_validator.recompute(seal, declared_inputs)
  REQUIRE distinct implementation, correct seal, valid certificate and pass

  IF registration is empirical:
    REQUIRE preregistered, isolated and custodially separated measurement
  ELSE:
    REQUIRE no empirical validator was silently added

  admitted := closure in {finite-complete, depth-independent}
  EMIT accepted receipt
  IF admitted:
    authority_ledger.admit(receipt)

```

Every REQUIRE is fail-closed. An exception raised by a claim callback is converted into a rejected receipt at the current gate. Later callbacks are not executed. Rejected and conditional evidence can be preserved, but the authority ledger accepts only an internally consistent receipt with both `accepted_evidence=true` and `model_admitted=true`.

The repository coordinator enforces the second authority boundary. It builds the source manifest, runs the engine, writes the receipt atomically, verifies the stored representation and only then permits a census update. When rebuilding authority, it reads the ordered census, verifies each stored receipt, checks dependency order and reconstructs the ledger from accepted receipts. A direct edit to `census/claims.json` cannot forge a working dependency because the next complete verification reruns the derivation and compares the recomputed receipt object with both the census hash and immutable receipt file.

13.1 Engine module responsibilities

Module	Responsibility and enforced boundary
<code>authority.py</code>	In-memory dependency authority populated only by admitted receipts; rejects forgery and conflict
<code>canonical.py</code>	Deterministic serialization and SHA-256 identities, including exact rational representation
<code>empirical.py</code>	Two-phase prediction envelope; refuses target access before seal
<code>engine.py</code>	Ordered fail-closed orchestration, violation checks and receipt construction
<code>errors.py</code>	Carries the complete rejected receipt when execution halts
<code>exact.py</code>	Refuses Boolean-as-count, nonpositive count, semantic zero, negative values, beyond-One parts and floating proof quantities
<code>external.py</code>	Separate-process validator adapter with source-manifest identity and strict result parsing

Module	Responsibility and enforced boundary
isolation.py	Capability and target-custody certificates without pretending Python is a universal sandbox
model.py	Immutable typed evidence records and derivation/validator protocols
portable.py	One macOS/Windows/Linux host vocabulary and a restricted child-process environment
publication.py	Branch-inventory and final-paper completeness gates
receipt_io.py	Atomic receipt persistence, exact-field parsing and hash verification
repository.py	Sole receipt-to-census path and authority reconstruction
source.py	Repository-contained source artifact hashing; rejects missing, empty or escaping files
__init__.py	Narrow public interface for the one engine package

14. Complete executable trace: the operational root theorem

This section reproduces the full public evidence chain for SFT-ROOT-THERE-IS-NO-NOTHING. The authoritative machine objects remain the JSON and Python artifacts named in the evidence map; the table below makes that chain readable without asking the reader to infer missing steps.

14.1 Registration trace

Property	Registered value
Claim	SFT-ROOT-THERE-IS-NO-NOTHING
Title	Premise-free operational root theorem
Branch	foundation
Evidence mode	formal
Statement	No admissible operational statement, denial, record, proof object or derivational object is nothing: absence presents no counterexample, while every presented counterexample is an occurrence.
Root theorems	empty tuple
Dependencies	empty tuple
Axioms	empty tuple
Free parameters	empty tuple
Provenance	direct_forcing
Excluded inputs	v2 source/certificates/answers; conventional ontology or logic as premise; application-selected rules; empirical targets
Source manifest	sha256:c3de6a8fc34d927dce243992a5a0de610f0c8701084ae08b3c9828838f75d915

The root exception is precise: every downstream claim must trace through exactly this one admitted root, while this claim must cite no root and no dependency. A denial is not imported as a logical axiom. It is submitted as an operational challenge to the generator.

14.2 Generator and completeness trace

Generation rule: partition every purported operational counterexample by whether it emits an identifiable presentation at the proof boundary.

Exact boundary: all operational statements, denials, records and proof challenges, independent of internal length or notation.

The classifier is a single property: presentation at the proof boundary. If no statement, denial, record, trace or identity is emitted, no counterexample object enters the proof boundary. If one is emitted, the emitted object is an occurrence. There is no third operational class relative to this classifier: any purported third challenge either emits an identifiable presentation and

is in the second class, or does not and supplies no proof artifact.

Expected cardinality is therefore the counted value two. The completeness certificate is
sha256:15bdb6e089afecc0a45498eabf39d25a1c93e2f5de8346fe998860bde750ed9d.

14.3 Generated candidates and decisions

Candidate	Exact form	Candidate trace	Decision and reason	Decision proof
unpresented-absence	No statement, denial, record, trace or identity is presented; there is therefore no operational counterexample to admit.	sha256:8b0f607cd624d7e45fb5af28484d0f9ae12f56636c09e7d0b72730f572a81d0a	Reject. Without a presentation there is no counterexample object to admit.	sha256:a05b0ae640948852199f68e8042506212c38e6d0c3bdc72a1c1c870029626eef
presented-occurrence	A statement, denial, record, trace or identity is presented; that presentation is an occurrence and therefore is not nothing.	sha256:6cb18c5ee96566de7b9e9a08633e48be1e3dd759029592085889f67fee04222	Survives. Presentation supplies the witnessed occurrence required by the theorem.	sha256:c6b4d78467ea08f053d781f0f024c4fbc5960b53eb8f4f044ebdb20811156044

Exactly one candidate survives. The conclusion is not selected by a target value; it follows from applying the same presentation rule to every member of the generated partition.

14.4 Minimality, uniqueness and generality

Minimality: removing the presented witness removes the only admissible proof object. What remains is not a smaller counterexample; it is the absence of a submitted challenge. Named-shape uniqueness: within the exhaustive presentation partition, only **presented-occurrence** enters execution, and its status as an occurrence prevents it from satisfying the proposed nothing class.

The reasoning depends only on presentation, not on content, notation, length, host encoding or internal depth. The exact boundary is therefore certified `depth_independent`. Closure proof:

sha256:b06285bf05b81202ccd4331e50c6d5e4f32db07d4f14aecbcb38ac8825ac7716. Generality certificate: sha256:67bee220b6f1c353142b2021a3c83640c1ed69235429155e6ebdf3bc1ee5a608.

14.5 Root adverse controls

Control	Deliberate challenge	Expected and observed result	Receipt identity
False premise	Treat unpresented absence as a presented counterexample	Expected rejection because no identity or trace exists; observed unpresented-absence not admissible	sha256:afe5d206d2c2de5d7ec6459854a1a4dec09310b2b3514f9569325262c70090a7
Tampered source	Alter registered derivation source identity	Expected rejection before enumeration; observed changed identity differs	sha256:4c7f233ed8e6c655aaf1094f2123f8642425ea92e9b9176dc8598b4adaccbc36
Tampered artifact	Mark both generated classes as surviving	Expected rejection for nonunique forcing; observed two-survivor set	sha256:a4a220c6532648805adcc82501fa683fc4c87572af689a0a9022abd97255fc0a
Boundary	Assert an unexpressed metaphysical object outside the operational boundary	Expected refusal because no checkable object is supplied; theorem remains operationally scoped	sha256:8d3a9ef27fade730c4c3484657e789118f8f2a67045e21fecc093da702fad8c0

14.6 Root seal, distinct recomputation and admission

The engine seals registration, two generated candidates, two decisions, closure and four controls as

sha256:19c2e2af082dc9777c13bafb3bc561c7fa5cb96a51905fb26cfa06552ebf998f. The separate validator implementation identity is

sha256:54ec0bbe6fecc15920c05fc3f75df6a66957c3c15a4076245ab22c36a7b766ca. It regenerates the presentation partition from declared inputs, recomputes the survivor and controls, and produces validation identity
sha256:7942e78a54a932551ad6f5a2ceb8c5fb9122dcae0f18510052d402f42c604e3b.

All eight receipt gates pass: registration, enumeration, forcing, form closure, controls, seal, distinct validation and model admission. The final receipt is
 sha256: 711864171e4d3a2f2734f0c2890965bcd81a0228349538751a3c80699c27d669. It contains no empirical validation because the registered claim is formal.

15. Complete executable trace: the structural One

SFT-FOUNDATION-ONE-001 is the first forward-forced result. It can run only after the root receipt has entered a fresh in-memory authority ledger.

15.1 Registration trace

Property	Registered value
Claim	SFT-FOUNDATION-ONE-001
Title	The structural One
Branch	foundation
Evidence mode	formal
Statement	The unique minimal representation of the admitted root occurrence is its complete self-whole with no unforced addition; this structural identity is the One.
Root trace	exactly SFT-ROOT-THERE-IS-NO-NOTHING
Dependencies	exactly SFT-ROOT-THERE-IS-NO-NOTHING
Axioms	empty tuple
Free parameters	empty tuple
Provenance	forward_forcing
Excluded inputs	v2 One definition/implementation; conventional numerical one; application-selected identity; measured magnitude
Source manifest	sha256:6ae512f49a7053affbd2587a859679ebc636a039a3e1d4a22f59a14f58c3a0f3

The result asks only how an admitted occurrence can be represented relative to retention of that occurrence and introduction of material not present in its sole dependency. It does not assume number, magnitude or arithmetic identity.

15.2 Generator and completeness trace

For root coverage, the generated relation has three exhaustive classes:

- **none**: none of the admitted identity is retained;
- **proper**: a strict fragment is retained, presupposing a cut; and
- **complete**: the admitted identity is retained as a whole.

For material beyond the root, the generated relation has two exhaustive classes: **no-extra** and **has-extra**. Their Cartesian product has counted cardinality six and exhausts representations classified solely by these two relations. The generator is not given the desired form name.

Completeness certificate:

sha256: 4d733e780b0e83a927e6c34268290ee82101f0a1f790e523fceb2ca3b581d892. Exact boundary: all representations classified solely by coverage of the admitted root occurrence and addition of material not supplied by it.

15.3 Six generated forms and full decision trace

Candidate	Generated relation	Candidate trace	Decision and reason	Decision proof
omitted-root	none coverage; no extra	sha256: ab3bf2a7990ce67f52063b78af5a48c211c4228cd408243dc087c2b275779d95	Reject: does not retain the admitted root occurrence	sha256: 638e651d319bde79d55c6c36849378872a99e0d48343c4b7caa6b85a93f151e

Candidate	Generated relation	Candidate trace	Decision and reason	Decision proof
replacement-extra	none coverage; has extra	sha256:76f402b7417cebd506e7cee8818524ebfaed73e244dde2cc30cd7c3a551a22f6	Reject: does not retain the admitted root occurrence	sha256:c6493c8c7ae0776d313afd11f6d7846b26db8dc2ea597568210531258b387ca1
fragmented-root	proper coverage; no extra	sha256:8adaa4ecace7f1914f47b40903a895113c25d44f928513ca29701f8787fad689	Reject: presupposes a cut and loses complete admitted identity	sha256:dab5ba1c22dca023d5c38434a0987490c3d1bb9525095ad2975d74df0ecac403
fragment-plus-extra	proper coverage; has extra	sha256:9f299064dc7fc26084c0cbeeda5a672213bdc3369f3f8fc78f8cd3799518b30d	Reject: presupposes a cut and loses complete admitted identity	sha256:5481c32d5600ff812dc3460e9796e26be160ef39a9b2c0441dc6a563732a9472
exact-self-whole	complete coverage; no extra	sha256:258126da54beeac598231fbe1bed20681ee32cbdd5bd601cf8b60af4075e92ea	Survives: retains the complete admitted occurrence without omission or addition	sha256:80937d3b539845ece0e167e0cafc8e3220609d0cd25e7c5670c71c856baa83d0
whole-plus-extra	complete coverage; has extra	sha256:01e607731f9aeabbd723352913692372bccac11fa801d9de825b64e9a229137c	Reject: introduces material absent from the sole admitted dependency	sha256:4b4a458070f0eb421838e93d79667908579afaae995913e359f88cf66183a024

Exactly one form is both complete and addition-free. Calling it the structural One records its relation to the admitted occurrence; it does not smuggle in a conventional numeral.

15.4 Minimality, uniqueness and generality

Deleting any retained root identity makes the candidate incomplete. Adding any material creates an occurrence not supplied by the sole dependency. Therefore `exact-self-whole` is minimal. Because every pair in the complete coverage and addition product was generated, and only (`complete`, `no-extra`) survives, the named shape is unique at the boundary.

The classifier does not inspect the root occurrence's content, notation, size or internal depth. It applies to an admitted identity as such. Closure is therefore depth-independent at the exact two-relation boundary. Closure proof:

sha256:87c7bd8ee1b561223c7d93309ebfd8662b89cff819f5a38b30f50dd5c48f5b27. Generality certificate: sha256:74e4c3761ec998027e1b12ed7c5f2581cd45132d11134cee15e13dffc8c64a19.

15.5 Structural One adverse controls

Control	Deliberate challenge	Expected and observed result	Receipt identity
False premise	Substitute an omitted root as a representation	Expected rejection; observed omitted-root does not survive	sha256:0dceced6a8585727559c36b83c2f4b0d0061703175b7ab66d4a653b3be67600a
Tampered source	Change source identity	Expected source refusal; observed identity differs	sha256:e4188d90cf7ccec55e447016899f3e85a391963b406e3ff5f914e99dcc3daaa3
Tampered artifact	Add material to the complete root	Expected rejection; observed whole-plus-extra does not survive	sha256:c857b9ae91fe5c805ba7e9d7b170f22ea1c25045938c3ae5c2599c5f948a199b
Boundary	Promote the result to a numerical magnitude	Expected refusal; observed meaning remains structural self-wholeness only	sha256:f9b13fb3dc82eeffcfa15517eb3320854a0b96759feb5e9846c0c352df66bce9

15.6 One seal, distinct recomputation and admission

Derivation seal: sha256:c62bb00cc30970ecf013bbc8225547a09395cf0719d3525f2a69bca1d582da7b.

Distinct validator implementation:

sha256:123f67f4797383feb1a659ec3b66812e0870e507a81ca0b061434bd273fad66b. Validation identity: sha256:ceaf767421b3450a862b10a6962da027c0467773a2e1be5115f554d2c3356083. Final receipt: sha256:68332624c276dc5d0ddfd568a26b95e20d7d5665fcd7b7e825ff1b7b19bf1d51c.

Registration, enumeration, forcing, form closure, controls, seal, distinct validation and model admission all pass. No empirical object is attached.

16. Verification and adverse-test census

The release suite contains 60 tests grouped by purpose rather than accumulated as an undifferentiated success number.

Test group	Count	Covered surfaces
Engine behavior	29	formal and empirical acceptance, conditional exclusion, axioms, parameters, enumeration, uniqueness, controls, independent identity, target order, capability policy, exact domain, publication gates, census authority, source drift and external process recomputation
Engine totality and malformed surfaces	18	forged receipts, canonical/exact guards, every invalid registration class, callback exceptions, malformed census/closure/control/external/empirical objects, isolation/custody fields, source escape, parser strictness and conflicting receipts
Structural One	4	six-form product, sole survivor, admitted-root dependency and tampered second survivor
Root theorem	4	premise-free registration, two-class partition, complete engine execution and tampered survivor
Repository scaffold	5	census identity, generation state, fail-closed policy, JSON parseability and required guidance

The public verifier launches the full suite under Python's standard-library `trace` module. It scans every `sft.engine` source module, counts executable lines, and fails on any line marked missing or any module not executed. The current count is 1,264 executed of 1,264 executable lines across 15 modules. Temporary trace files are discarded after the run.

After software verification, the command loads `census/execution_manifest.json`. It requires the manifest claim order to match `census/claims.json` exactly. It then creates a blank authority ledger, loads each claim factory from its registered execution file, rebuilds the source manifest, reruns the engine and distinct validator, compares the entire newly computed receipt to the immutable stored receipt, and admits it to the fresh ledger before executing its dependants. No verification run writes scientific evidence.

The suite deliberately tests unfavorable paths, including:

- axiom or parameter present at registration;
- missing, duplicate or reordered candidates and decisions;
- no survivor or multiple survivors;
- failed or duplicate controls;
- forged, incomplete, extra-field or hash-mismatched receipts;
- same implementation presented as independent;
- validator subprocess failure or malformed output;
- source file missing, empty or outside the repository;
- target access before seal, wrong target identity and duplicate seal;
- forbidden prediction capability and mismatched custodian;
- conditional result attempting to become a dependency;
- branch paper with an open frontier; and
- final paper without every completed branch paper.

These controls demonstrate that named failures are detected. They do not prove that the test author has imagined every future failure mode.

17. Threat model, portability and empirical custody

17.1 Threats addressed

The engine is designed against silent answer import, post hoc result selection, incomplete candidate presentation, source drift, artifact tampering, forged status, dependency bypass, target leakage, selective reporting and publication before inventory completion. Each threat is mapped to an executable gate and a retained identity.

17.2 Threats not claimed solved

No hash system protects a compromised machine that changes both program and reported evidence before any honest copy exists. No Python process can universally sandbox hostile native code across all operating systems. No mechanical schema can decide whether a scientific grammar omitted a concept the author failed to generate. No internal validator equals independent community replication. These are disclosed review surfaces, not hidden assumptions.

The publication response is redundancy: public git history, DOI archives, checksums, separate-process validators, future generated-C or SFT-language certificates, independent forks and explicit empirical custody. A contradiction between copies becomes observable.

17.3 Cross-platform contract

The runtime is standard-library-first Python and recognizes exactly `macos`, `windows` and `linux` as receipt metadata. Host identity cannot select a scientific branch. Child validator processes receive only a controlled minimal environment rather than arbitrary parent variables. The same engine policy and candidate logic run on each platform.

The repository offers one logical verifier, `sft verify-all`; the interpreter launcher is spelled `python3`, `python` or `py` according to a host's Python installation. Docker, a virtual machine and cloud execution are not required.

17.4 Future official empirical run

An official natural-science prediction requires two custodians or equivalently separated roles. The prediction custodian receives the registered program, allowed inputs and only target identities. The capability-closed interpreter has no instruction for clock, environment, filesystem, network, subprocess, dynamic import or foreign function access. Its certificate binds interpreter, program, inputs, denied capabilities, trace and proof of target absence.

The target custodian binds the same experiment registration and target identities, retains target content, and produces a cross-bound certificate that the data were not released before the prediction seal. Only then may the evaluator verify the seal, open the target, calculate preregistered measurements and emit the complete-row receipt. A favorable score without those objects may support an instrument-performance report but cannot enter SFT as a closed law.

18. Full paper-to-evidence map

Every derivational statement in this paper maps to public artifacts. Relative paths are stable inside the repository and Zenodo bundle.

Paper subject	Registration and generator	Decisions and closure	Independent proof and receipt
Operational root theorem	<code>claims/SFT-ROOT-THERE-IS-NO-NOTHING/registration.json</code> ; <code>candidate_census.json</code> ; <code>sft/foundation/root_theorem.py</code>	<code>elimination_receipt.json</code> ; <code>controls.json</code> ; <code>STATUS.md</code>	<code>independent_validator.py</code> ; <code>certificate.json</code> ; <code>receipts/engine/model_admitted/SFT-ROOT-THERE-IS-NO-NOTHING-711864171e4d3a2f.json</code>
Structural One	<code>claims/SFT-FOUNDATION-ONE-001/registration.json</code> ; <code>candidate_census.json</code> ; <code>sft/foundation/one.py</code>	<code>elimination_receipt.json</code> ; <code>controls.json</code> ; <code>STATUS.md</code>	<code>independent_validator.py</code> ; <code>certificate.json</code> ; <code>receipts/engine/model_admitted/SFT-FOUNDATION-ONE-001-68332624c276dc5d.json</code>

Paper subject	Registration and generator	Decisions and closure	Independent proof and receipt
Engine policy	CONSTITUTION.md; governance/engine_policy.json; schemas under governance/	sft/engine/engine.py; model.py; exact.py; isolation.py	tests/test_engine.py; tests/test_engine_totality.py; docs/VERIFICATION.md
Census authority	census/execution_manifest.json; census/claims.json	sft/engine/repository.py; receipt_io.py; authority.py	sft/verification.py; model-admitted receipt directory
Empirical method	docs/EMPIRICAL_METHOD.md; experiment schema and registration template	sft/engine/empirical.py; isolation.py	Current state: infrastructure tested, no official v3 empirical claim admitted
Prior-work supersession	prior-work-ledger/manifest.json; NOTICE.md	Post-seal correspondence files	V2 commit and SHA-256 identities recorded in the ledger
Publication constitution	publications/BRANCH_PAPER_PROTOCOL.md; FINAL_TOE_PAPER_PROTOCOL.md	sft/engine/publication.py	Publication failure tests in engine suite
Rights and participation	LICENSE; LICENSING.md; TRADE MARK_POLICY.md; CONTRIBUTING.md	governance/ERNOS_LABS.md	Public repository history and archived DOI edition

The two `WHY_DERIVATION_CHECK.md` files provide a cold-readable bridge from the formal artifacts to human interpretation. They are guidance, not additional premises. `correspondence/foundation/` records historical comparison only after the v3 seals. The v2 source files recorded in the prior-work ledger have hashes `sha256:0bf79de6c3f42784424537481daf9df310c71cc1c502c148e6aa27bcd1f2a401` for `STANDARDS.md` and `sha256:42c4be709dcd9edcfbedc70ee82055a8660d9658de21758561fd46e068a727bf` for `OneFoldMaster.md`; neither is in a v3 dependency tuple.

19. Conclusion

Ernos Labs is an attempt to restore empirical primacy and public reproducibility at a scale large enough for an ambitious unification programme. It does not ask readers to accept SFT because of an institution, a credential, a consensus vote, a funding decision or an oracle's score. It asks them to inspect a finite claim, run its construction, attack its boundary, reproduce its receipt and preserve the outcome.

The present result is intentionally small: one premise-free operational root, one forced structural whole, one public admission engine and one open knowledge tree. Its significance, if any, will be earned only by whether the same method continues to close later branches without importing their answers—and whether independent researchers can prove where it succeeds or fails.

Data, code, licenses and citation

All source, candidate censuses, decisions, controls, certificates and receipts for this paper are contained in this repository and archival release bundle. Code and generated software artifacts are licensed under the Apache License 2.0. This paper, documentation and non-code evidence are licensed under Creative Commons Attribution 4.0 International. Maria Smith retains copyright. Third-party references retain their own rights.

Use `[CITATION.cff](CITATION.cff)` for software citation and cite this paper as Smith M. (2026), *There Is No Nothing: A Premise-Free Operational Foundation and an Open Verification Platform for Smithian Fold Theory*, [doi:10.5281/zenodo.21514890](https://doi.org/10.5281/zenodo.21514890). Cryptographic checksums are provided in the release bundle.

References

1. UNESCO. *Recommendation on Open Science*. 2021. [Official text](#).
2. National Academies of Sciences, Engineering, and Medicine. *Reproducibility and Replicability in Science*. 2019. [doi:10.17226/25303](#).
3. Lundh A, Lexchin J, Mintzes B, Schroll JB, Bero L. Industry sponsorship and research outcome: systematic review with meta-analysis. *Intensive Care Medicine*. 2018;44:1603–1612. [doi:10.1007/s00134-018-5293-7](#).
4. Fabbri A, Lai A, Grundy Q, Bero LA. The influence of industry sponsorship on the research agenda: a scoping review. *American Journal of Public Health*. 2018;108:e9–e16. [PubMed](#).
5. Gallo SA et al. Ranking versus rating in peer review of research grant applications. *PLOS ONE*. 2023. [Full text](#).
6. Demicheli V, Di Pietrantonj C. Peer review for improving the quality of grant applications. *Cochrane Database of Systematic Reviews*. [Full text](#).
7. Bernaldo A et al. Ending publication bias: a values-based approach to surface null and negative results. *Nature Communications*. 2025. [Full text](#).
8. National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework 1.0: AI risks and trustworthiness*. 2023. [Official text](#).
9. Heil BJ et al. Reproducibility standards for machine learning in the life sciences. *Nature Methods*. 2021;18:1132–1135. [doi:10.1038/s41592-021-01256-7](#).
10. National Institutes of Health. *Rigor and Reproducibility*. [Official policy resource](#).

Copyright © 2026 Maria Smith. Code: Apache-2.0. Paper and documentation: CC BY 4.0. “Ernos Labs” is a reserved conformance designation; see ``TRADEMARK_POLICY.md``.